Atividade 3 – RBFNet com Duplicatas e Regularização

Aprendizado Computacional e Reconhecimento de Padrões

Caique Frederico Pereira de Moura

23 de abril de 2026

Sumário

1 Objetivo da atividade **2**

2 Tópico 1 – RBFNet com duplicatas **2**
2.1 Problema numérico 2
2.2 Estratégia proposta 2
2.3 Implementação 2

3 Tópico 2 – PolyMap-RBFNet-SVMLin **2**
3.1 Etapas do método 2
3.2 Resumo da implementação 3
3.3 Resultados gráficos 3

4 Tópico 3 – Aplicação em Wine, Iris e base do Exercício 1 **5**
4.1 Objetivo 5
4.2 Heatmaps SVM-RBF 5
4.3 Heatmaps Poly-RBF-SVM 7

5 Resultados e discussão **8**

6 Conclusão **8**

1 Objetivo da atividade

Este relatório apresenta a implementação e a análise de duas abordagens: (i) RBFNet com regularização para lidar com duplicatas na base de treino e (ii) pipeline **PolyMap-RBFNet-SVMLin**, com comparação sistemática entre bases e métodos.

2 Tópico 1 – RBFNet com duplicatas

2.1 Problema numérico

No treinamento supervisionado da RBFNet com erro quadrático (SSE), os pesos da camada de saída podem ser obtidos pela solução fechada:

$$\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}, \tag{1}$$

em que Φ é a matriz de ativações da camada RBF. Quando há padrões idênticos na base, colunas/linhas de Φ podem ficar linearmente dependentes, tornando $\Phi^T \Phi$ singular ou mal condicionada.

2.2 Estratégia proposta

A estratégia adotada foi a **regularização de Tikhonov (Ridge)** na etapa supervisionada:

$$\mathbf{w} = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \mathbf{y}, \quad \lambda > 0. \tag{2}$$

Essa modificação desloca os autovalores da matriz normal e reduz o número de condição, aumentando a estabilidade numérica sem alterar a estrutura da RBFNet.

2.3 Implementação

Foi usado o conjunto artificial solicitado:

```
X, y = make_moons(n_samples=500, noise=0.1, random_state=42)
X = np.vstack([X, X[:20]])
y = np.hstack([y, y[:20]])
```

No notebook, a classe RBFNet foi implementada com centros por KMeans e estimação regularizada dos pesos:

```
I = np.eye(Phi.shape[1])
self.weights = np.linalg.inv(Phi.T @ Phi + self.lambd * I) @ Phi.T @ y
```

Configuração principal do experimento: $n_centers = 15$, $\sigma = 0.5$, divisão treino/teste 70/30 e $\lambda = 10^{-2}$ para o modelo final.

3 Tópico 2 – PolyMap-RBFNet-SVMLin

Nesta etapa, foi implementado o processo combinado **PolyMap-RBFNet-SVMLin**, seguindo exatamente as quatro fases solicitadas.

3.1 Etapas do método

1. Remapeamento polinomial: os dados originais em $\mathbb{R}^n$ são transformados para $\mathbb{R}^q$ via `PolynomialFeatures(degree=d)`.

2. Camada RBF (fase não supervisionada): com os dados remapeados, define-se o número de centros como

$$N_c = d \cdot q, \tag{3}$$

   e os centros são obtidos por KMeans.

3. Classificação linear: as ativações RBF alimentam um SVM linear com penalidade C e estratégia OVR.

4. Avaliação: o desempenho final é medido pelo percentual de acerto (acurácia).

3.2 Resumo da implementação

No notebook, o fluxo foi implementado por uma função no formato:

```
def poly_rbf_svm(X, y, d=2, C=1.0, sigma=1.0):
        X_train, X_test, y_train, y_test = train_test_split(
                X, y, test_size=0.3, random_state=42
        )

        poly = PolynomialFeatures(degree=d)
        X_train_poly = poly.fit_transform(X_train)
        X_test_poly = poly.transform(X_test)

        q = X_train_poly.shape[1]
        n_centers = d * q

        centers = KMeans(n_clusters=n_centers, random_state=42).fit(
                X_train_poly
        ).cluster_centers_

        Phi_train = rbf(X_train_poly, centers, sigma)
        Phi_test = rbf(X_test_poly, centers, sigma)

        svm = LinearSVC(C=C, max_iter=10000)
        svm.fit(Phi_train, y_train)

        y_pred = svm.predict(Phi_test)
        return accuracy_score(y_test, y_pred)
```

3.3 Resultados gráficos

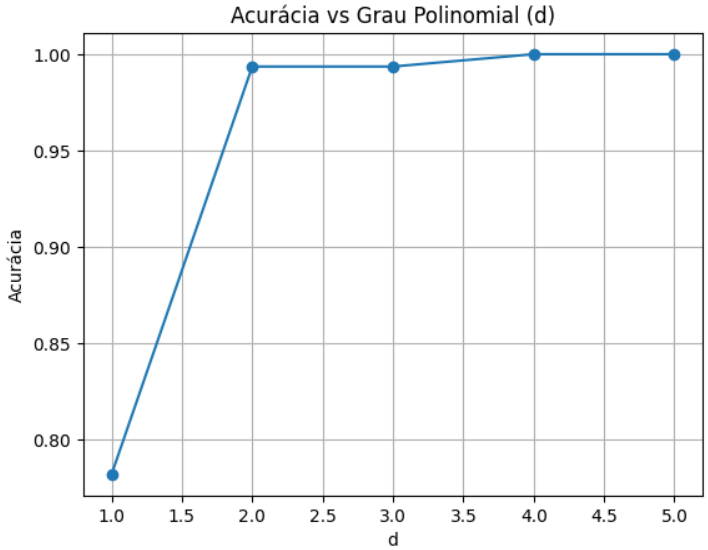

Figura 1: Acurácia em função do grau polinomial d.

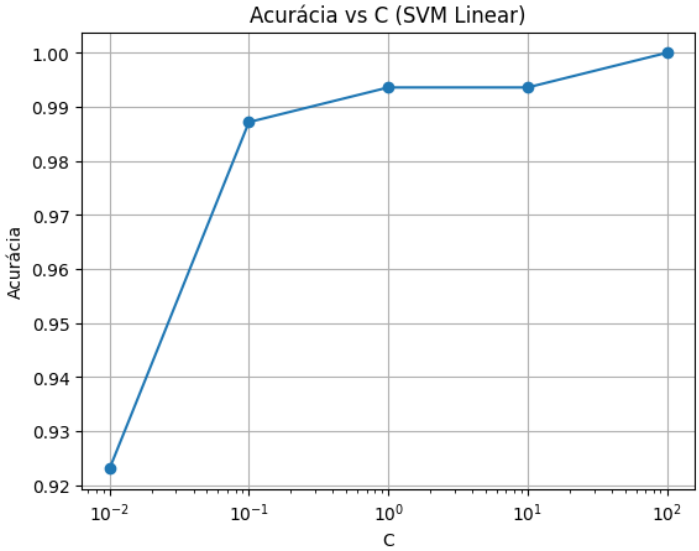

Figura 2: Acurácia em função de C no SVM linear.

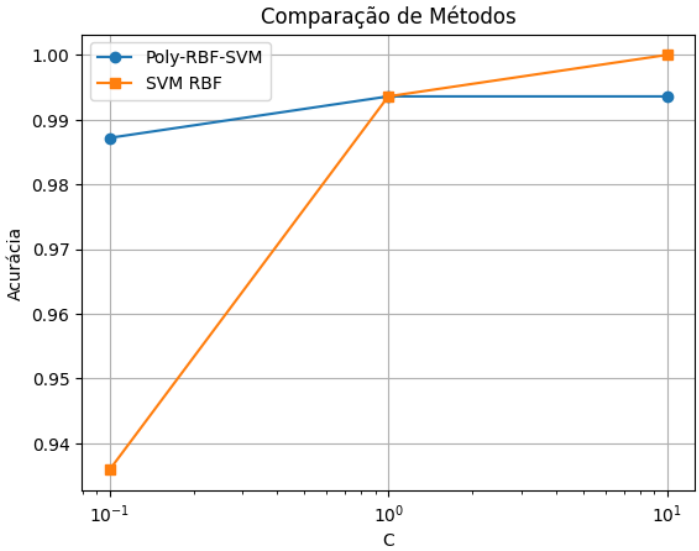

Figura 3: Comparação entre PolyMap-RBFNet-SVMLin e SVM-RBF.

4 Tópico 3 – Aplicação em Wine, Iris e base do Exercício 1

4.1 Objetivo

Aplicar o método **PolyMap-RBFNet-SVMLin** às bases *Wine Dataset*, *Iris Dataset* e à base sintética do Tópico 1, analisando o efeito dos parâmetros d e C. Em paralelo, comparar os percentuais de acerto com o método SVM com kernel RBF para diferentes valores de C e γ.

4.2 Heatmaps SVM-RBF

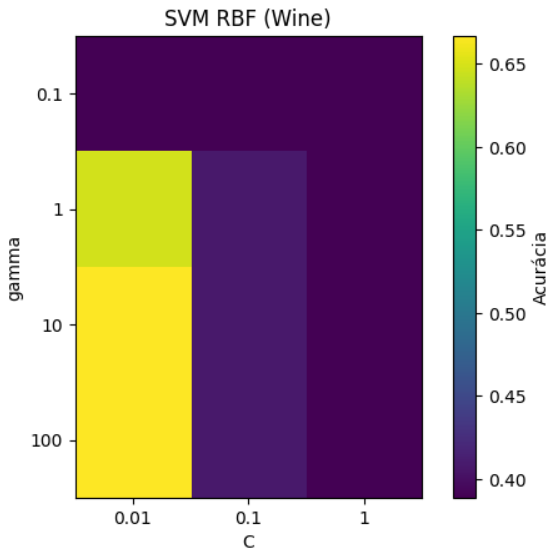

Figura 4: Heatmap de acurácia do RBF-SVM na base Wine - Não normalizado.

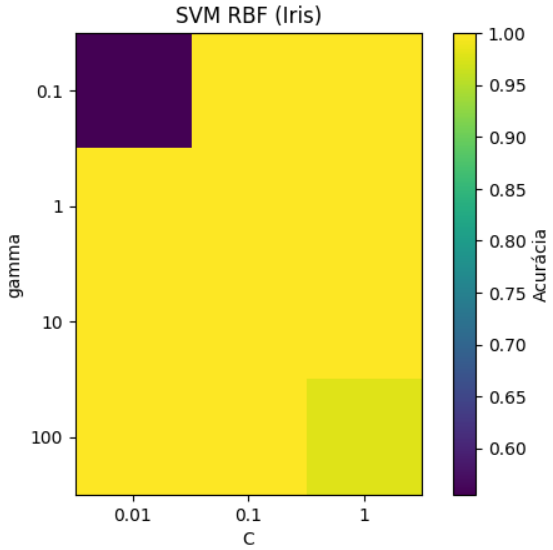

Figura 5: Heatmap de acurácia do RBF-SVM na base Iris.

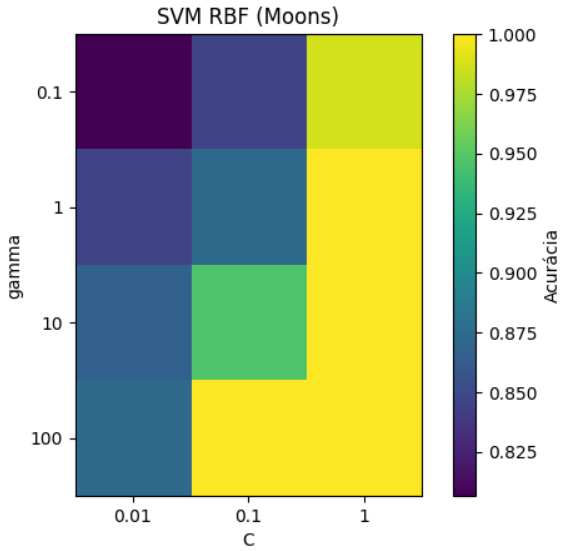

Figura 6: Heatmap de acurácia do RBF-SVM na base sintética (Moons).

4.3 Heatmaps Poly-RBF-SVM

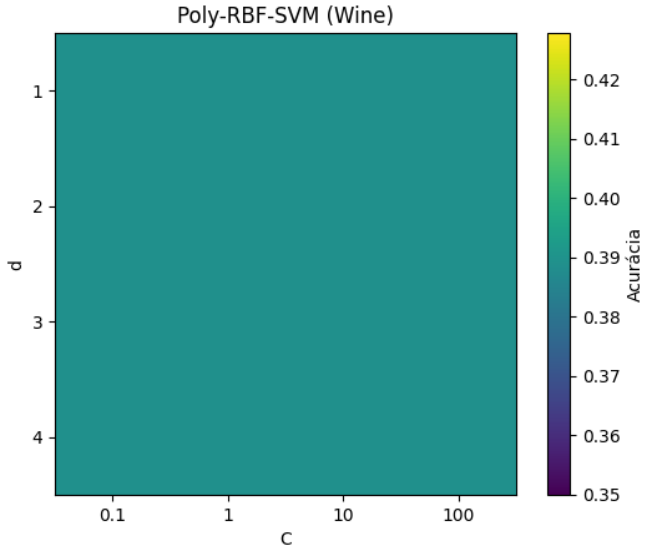

Figura 7: Heatmap de acurácia do Poly-RBF-SVM na base Wine - Não normalizado.

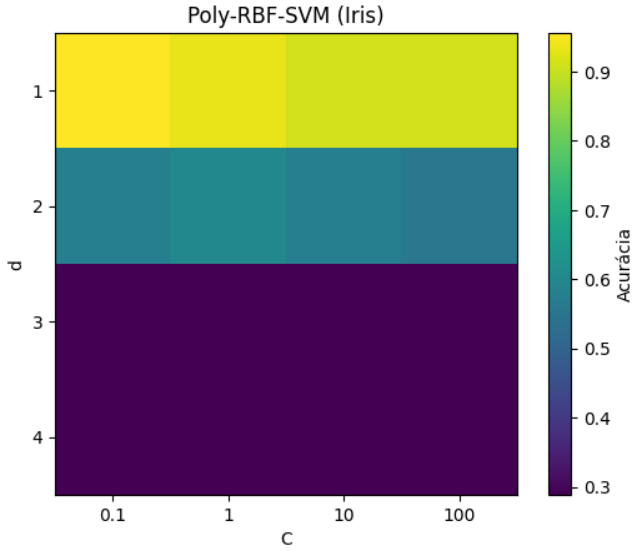

Figura 8: Heatmap de acurácia do Poly-RBF-SVM na base Iris.

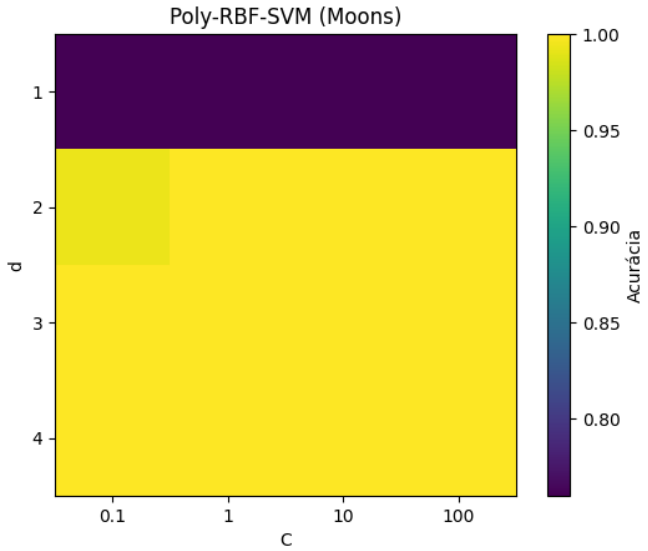

Figura 9: Heatmap de acurácia do Poly-RBF-SVM na base sintética (Moons).

5 Resultados e discussão

Considerando o **Tópico 1**, a regularização Ridge mostrou-se efetiva para estabilizar o treinamento supervisionado da RBFNet na presença de amostras repetidas. Mesmo com padrões idênticos na base, a modelagem permaneceu numericamente estável, com redução da sensibilidade dos pesos e manutenção de alto desempenho preditivo.

No **Tópico 2**, os gráficos de variação de d e C indicam que a abordagem PolyMap-RBFNet-SVMLin é sensível à escolha de hiperparâmetros, mas apresenta comportamento consistente em faixas intermediárias. A análise visual por curvas e heatmap reforça que a combinação entre remapeamento polinomial e camada RBF pode ampliar a capacidade discriminativa quando os parâmetros são bem ajustados.

No **Tópico 3**, a comparação entre bases (Wine, Iris e sintética) mostra que o método proposto mantém desempenho competitivo frente ao SVM com kernel RBF. Em termos práticos, o SVM-RBF tende a apresentar maior robustez global, enquanto o PolyMap-RBFNet-SVMLin oferece maior flexibilidade na construção explícita do espaço de características, o que pode ser vantajoso para análise e controle do modelo, porém com maior custo de ajuste.

De forma integrada, os resultados sugerem um compromisso entre **estabilidade e simplicidade** (SVM-RBF) e **flexibilidade de representação** (PolyMap-RBFNet-SVMLin), com desempenho final dependente do cenário e da calibração de d, C e γ.

6 Conclusão

Observa-se que o método PolyMap-RBFNet-SVMLin apresenta desempenho competitivo em relação ao SVM com kernel RBF, especialmente para valores intermediários de d e C. No entanto, o SVM RBF tende a ser mais estável, uma vez que seu kernel já incorpora implicitamente transformações não lineares. Por outro lado, a abordagem proposta permite maior controle sobre a construção do espaço de características, ao custo de maior complexidade e sensibilidade à escolha dos parâmetros.